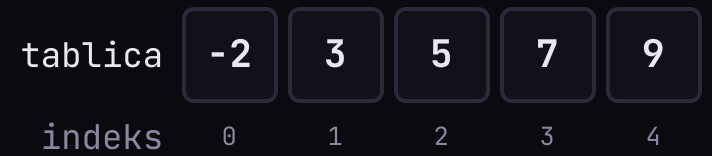
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Tablice w C++ — krok po kroku

Wszystko o zwykłych tablicach, powoli i po kolei: jak stworzyć tablicę, co w niej siedzi na starcie, jak numerujemy jej elementy, jak ją wypisać, wypełnić i wczytać. Każdy krok na osobnym slajdzie — z przykładem kodu i tym, co pojawi się na ekranie.

C++ · LEKCJA 7 · TABLICE · OD ZERA



PLAN PREZENTACJI

01 Po co nam tablica?
02 Tworzenie tablicy
03 Rozmiar policzy C++
04 Tablica bez listy
05 Indeksowanie
06 Odczyt jednego elementu
07 Zmiana jednego elementu
08 Wypisywanie całej tablicy
09 Ustawianie wartości
10 Wczytywanie tablicy
11 Sprawdź się
12 Najczęstsze błędy
13 Podsumowanie

01 Problem: dużo liczb naraz

Do tej pory każdą liczbę trzymaliśmy w **osobnej zmiennej**. Gdy liczby są trzy — da się wytrzymać:

3 LICZBY — 3 ZMIENNE

```
int a = 4;
int b = 7;
int c = 1;
```

A gdyby liczb było 5? 100? **1000**? Wymyślanie tysiąca nazw i pisanie tysiąca linijek nie ma sensu.

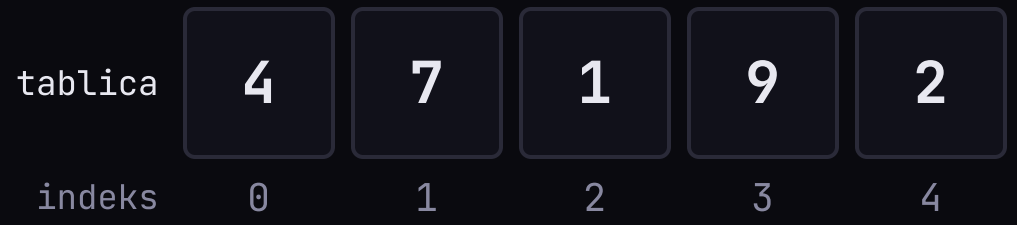
1000 LICZB — 1000 ZMIENNYCH?

```
int a1 = 4;
int a2 = 7;
int a3 = 1;
int a4 = 9;
int a5 = 2;
// ... i tak dalej ...
int a1000 = 6;   // 1000 linijek?!
```

Wniosek: Potrzebujemy **jednej nazwy** na **wiele liczb**. Do tego służy **tablica**.

01 Tablica = rząd ponumerowanych szufladek

Tablica to **jedna zmienna**, która ma w środku **wiele miejsc** na liczby — jak rząd szufladek ustawionych jedna obok drugiej.



1. Cała tablica ma **jedną nazwę** — tutaj `tablica`.
2. Każda szufladka trzyma **dokładnie jedną liczbę**.
3. Wszystkie szufladki mają **ten sam typ** — tutaj każda to `int`, czyli liczba całkowita.
4. Szufladki są **ponumerowane**. Numer szufladki to **indeks**. Zauważ, że numeracja zaczyna się od **0** — dokładnie omówimy to w rozdziale 05.

01 Gdzie piszemy kod? — szablon programu

SZABLON KAŻDEGO PROGRAMU

```
#include <iostream>     // 1. biblioteka z cin i cout
using namespace std;    // 2. cout zamiast std::cout

int main(){             // 3. tu zaczyna się program
    // <- tu wpisujemy kod z tablicami
}                       // 4. tu program się kończy
```

Każdy przykład z tej prezentacji to kawałek **zwykłego programu** w C++.

Żeby slajdy były czytelne, często pokazujemy **tylko ważne linijki** — resztę programu (szablon obok) dopisujesz sam.

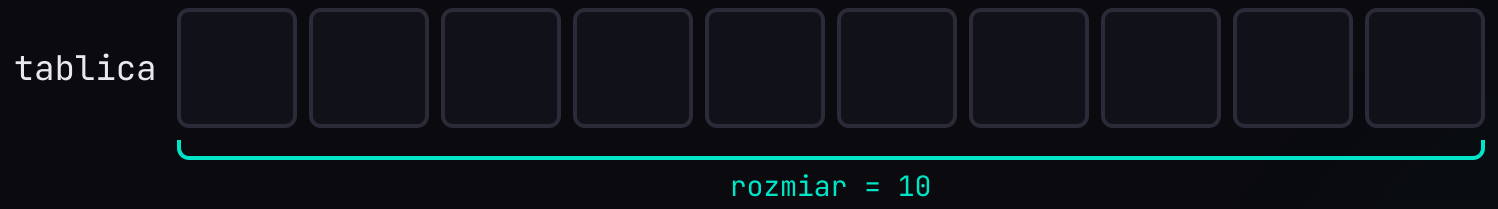
> Jeśli na slajdzie widzisz sam fragment, np. `int tablica[5] = {-2, 3, 5, 7, 9};`, to w prawdziwym programie stoi on **w środku** `main()` — w podświetlonej linijce. Wyjątek to tablice **globalne** (rozdział 04) — te piszemy **nad** main.

02 Najważniejsza zasada tablic

Na początku ustalamy wielkość tablicy i później nie możemy tego zmienić.

Wielkość tablicy (inaczej: **rozmiar**) to liczba szufladek. Wybieramy ją **raz** — w chwili tworzenia tablicy. Potem tablica już **nie urośnie i nie zmaleje**.

```
int tablica[10];   // 10 szufladek — na zawsze
```



02 Budowa linijki, która tworzy tablicę

Rozłóżmy na części jedną linijkę. Każdy kawałek ma swoje zadanie:

```
int tablica[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

1 `int` — **typ** — co leży w szufladkach. int = liczby całkowite.

2 `tablica` — **nazwa** — wymyślamy ją sami (np. tablica, lista1, liczby).

3 `[10]` — **rozmiar** w nawiasach kwadratowych — ile jest szufladek.

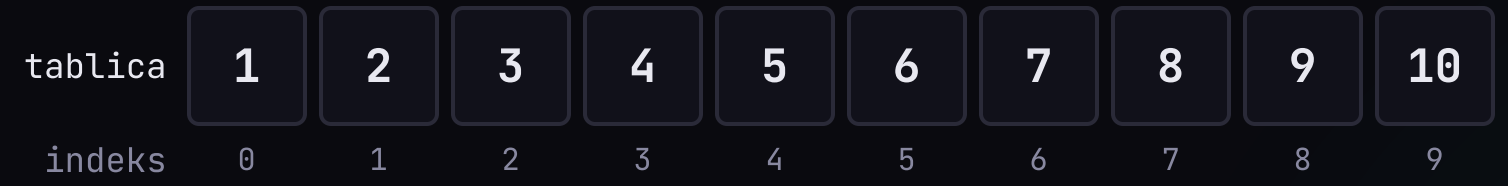
4 `= {1, 2, …, 10}` — **lista inicjalizacyjna** — co wpisać do szufladek: w klamrach, po przecinku, po kolei.

5 `;` — **średnik** na końcu — jak po każdym poleceniu.

02 Przeczytajmy to po polsku

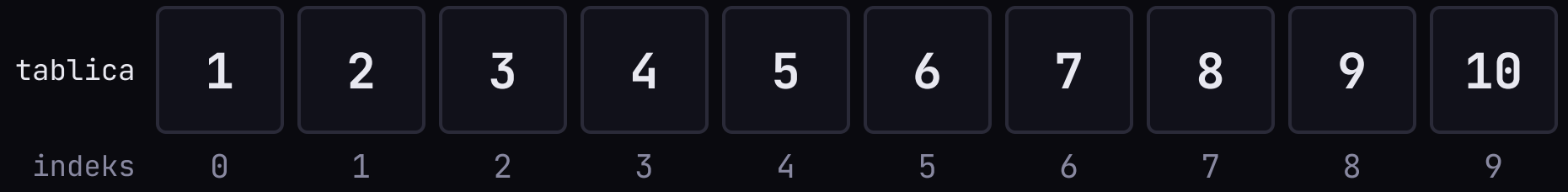
```
int tablica[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

1. `int` → „Zrób miejsce na **liczby całkowite**…”
2. `tablica` → „…nazwij je **tablica**…”
3. `[10]` → „…niech będzie w nim **10 szufladek**…”
4. `= {1, 2, …, 10}` → „…i wpisz do nich **po kolei** liczby 1, 2, 3, …, 10.”



02 Przykład 1: tablica z 10 liczbami

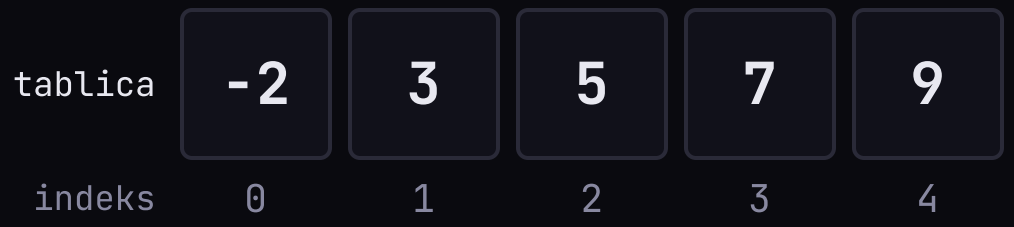
```
int tablica[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```



W nawiasie **[10]** — 10 szufladek. W klamrach — **10 liczb**. Liczby trafiają do szufladek **w tej samej kolejności**, w jakiej je napisaliśmy: pierwsza liczba do pierwszej szufladki, druga do drugiej i tak dalej.

02 Przykład 2: liczby mogą być ujemne

```
int tablica[5] = {-2, 3, 5, 7, 9};
```

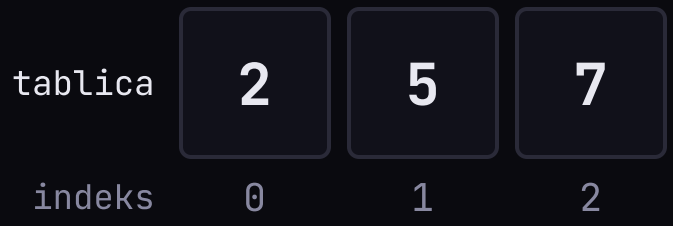


Rozmiar **5** → 5 szufladek → w klamrach **5 liczb**.

> Typ `int` mieści też liczby **ujemne**, więc **−2** to zwykły element. Liczby nie muszą też rosnąć ani iść po kolei — tablica zapamiętuje **dokładnie to**, co wpiszemy.

02 Przykład 3: mała tablica

```
int tablica[3] = {2, 5, 7};
```

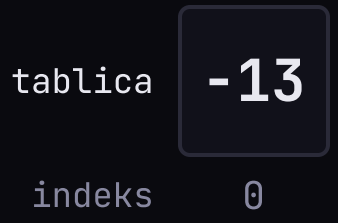


Trzy szufladki, trzy liczby: **2**, **5** i **7**. Tablica może być dowolnie mała.

> **Sprawdzaj zawsze:** **ile liczb w klamrach** = **rozmiar w nawiasie**. Tutaj: 3 liczby i [3]. Zgadza się.

02 Przykład 4: tablica z jednym elementem

```
int tablica[1] = {-13};
```



Tablica może mieć nawet **jedną** szufladkę. To nadal tablica — po prostu z jednym miejscem. W klamrach jest jedna liczba: **-13**.

> Nawet przy jednej liczbie piszemy klamry: `{-13}`.

02 Cztery przykłady obok siebie

Linijka	Rozmiar	Co jest w szufladkach
`int tablica[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};`	10	1 2 3 4 5 6 7 8 9 10
`int tablica[5] = {-2, 3, 5, 7, 9};`	5	-2 3 5 7 9
`int tablica[3] = {2, 5, 7};`	3	2 5 7
`int tablica[1] = {-13};`	1	-13

To są **cztery osobne przykłady**. W jednym programie dwie tablice **nie mogą** mieć tej samej nazwy — gdy potrzebujesz kilku tablic naraz, nazwij je różnie, np. `lista1`, `lista2`.

02 Rozmiaru nie da się zmienić

```
int tablica[5] = {-2, 3, 5, 7, 9};

// chcemy dopisać szóstą liczbę, np. 11...
// ale tablica ma 5 szufladek
// i ZAWSZE będzie miała 5.
// Nie ma polecenia, które dorobi szóstą.
```



Szóstej szufladki **nie ma** i nie da się jej dorobić.

Sposób: A co, gdy nie wiemy z góry, ile będzie liczb? Bierzemy tablicę **z zapasem** — większą, niż trzeba. Pokażemy to dokładnie w rozdziale 10 (wczytywanie).

03 Lista podana? Rozmiar można pominąć

Jak podczas tworzenia podajemy, na co ustawiamy szufladki — czyli podajemy **listę inicjalizacyjną** — to **nie musimy pisać rozmiaru**, ponieważ C++ sam się domyśli.

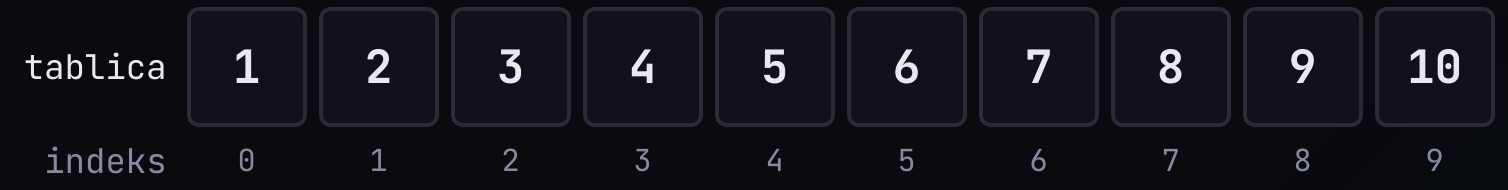
Z ROZMIAREM

```
int tablica[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

BEZ ROZMIARU — TO SAMO!

```
int tablica[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

Puste nawiasy `[]` znaczą: **„C++, policz sam”**. C++ liczy liczby w klamrach — jest ich 10 — więc tablica dostaje rozmiar 10. Obie linijki tworzą **identyczną** tablicę:



03 Jak C++ liczy rozmiar? Krok po kroku

```
int tablica[] = {-2, 3, 5, 7, 9};
```

1. C++ widzi puste nawiasy `[]` — rozmiaru nie podano.
2. Zagląda do klamer i **liczy** liczby: -2 to 1., 3 to 2., 5 to 3., 7 to 4., 9 to 5.
3. Wychodzi **5** — więc tworzy tablicę z **5 szufladkami**.
4. Wpisuje do nich liczby po kolei — tak samo jak zawsze.



Czyli: Efekt jest taki sam, jak po napisaniu `int tablica[5] = {-2, 3, 5, 7, 9};`

03 Te same cztery przykłady — bez rozmiaru

Linijka bez rozmiaru	C++ liczy	Wychodzi to samo co
`int tablica[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};`	10 liczb	`int tablica[10] = …`
`int tablica[] = {-2, 3, 5, 7, 9};`	5 liczb	`int tablica[5] = …`
`int tablica[] = {2, 5, 7};`	3 liczby	`int tablica[3] = …`
`int tablica[] = {-13};`	1 liczba	`int tablica[1] = …`

Który zapis wybrać? **Oba są poprawne.** Bez rozmiaru jest wygodniej — nie trzeba liczyć samemu, więc nie da się pomylić.

03 Uwaga: puste [] tylko razem z listą!

ŹLE — ANI ROZMIARU, ANI LISTY

```
int tablica[];   // BŁĄD!
```

C++ nie ma **z czego policzyć** rozmiaru — klamer nie ma. Program się **nie skompiluje**.

DOBRZE — ROZMIAR PODANY

```
int tablica[5];
```

DOBRZE — C++ POLICZY Z LISTY: 3

```
int tablica[] = {1, 2, 3};
```

Tablica **bez listy** (jak `int tablica[5];`) to temat następnego rozdziału — bo wtedy pojawia się ważne pytanie: **co jest w środku**?

04 Możemy też nie podawać listy

Listę inicjalizacyjną można pominąć. Wtedy piszemy tylko typ, nazwę i rozmiar:

```
int lista1[5];
```



Tablica ma 5 szufladek… ale **co w nich jest**? Nie wpisaliśmy żadnych liczb.

Odpowiedź zależy od tego, **gdzie** stworzymy tablicę. Mamy **dwa przypadki**:

`Przypadek nr 1 — tablica globalna`

same zera: **{0, 0, 0, 0, 0}**

`Przypadek nr 2 — tablica lokalna`

losowe śmieci, np. **{-5, 123, 15, 3, -10}**

04 Gdzie jest „globalnie”, a gdzie „lokalnie”?

STREFA GLOBALNA — POZA MAIN

```
#include <iostream>
using namespace std;

int lista1[5];      // globalna

int main(){
    STREFA LOKALNA — W KLAMRACH MAIN
    int lista2[5];  // lokalna
}
```

Globalna

stworzona **poza main**, poza innymi funkcjami, poza klamrami `{ }`. Zwykle na górze pliku, nad main.

Lokalna

stworzona **wewnątrz main** albo innej funkcji — czyli **w środku klamer** `{ }`.

Podpowiedź: Patrz na klamry. Jeśli tablica stoi w środku klamer jakiejś funkcji — jest **lokalna**.

04 Przypadek nr 1: tablica globalna

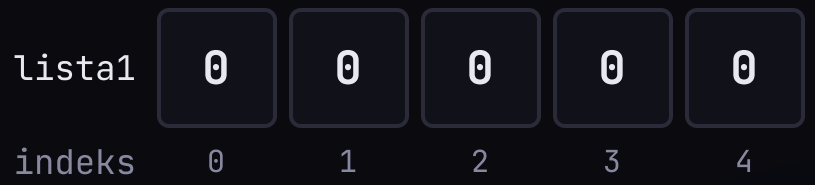
```cpp
#include <iostream>
using namespace std;

int lista1[5];   // poza main → globalna

int main(){

}
```

Gdy tworzymy tablicę **globalną** (poza main, poza innymi funkcjami, poza klamrami), to ta tablica **domyślnie jest ustawiana na same zera**.



Zapamiętaj: globalna tablica bez listy = **{0, 0, 0, 0, 0}**. Zawsze — przy każdym uruchomieniu programu.

04 Przypadek nr 2: tablica lokalna

```cpp
#include <iostream>
using namespace std;

int main(){
    int lista2[5];   // w main → lokalna
}
```

Gdy tworzymy tablicę **lokalną** (wewnątrz main albo innej funkcji, w klamrach), to ta tablica **domyślnie jest ustawiana na losowe śmieci**, np.:



Uwaga: te liczby to **przypadkowe resztki** w pamięci komputera. Przy następnym uruchomieniu mogą być zupełnie inne. **Nie wolno na nich polegać.**

04 Skąd się biorą śmieci?

1. Program tworzy lokalną tablicę `lista2[5]` i dostaje na nią **5 miejsc w pamięci**.
2. W tych miejscach **coś już wcześniej było** — liczby zostawione przez inne części programu.
3. C++ — żeby działać szybciej — **nie czyści** tych miejsc przy tworzeniu lokalnej tablicy.
4. W szufladkach zostaje to, co tam leżało. To właśnie **śmieci**.
5. Tablice **globalne** C++ czyści zawsze — dlatego tam są zera.

Ten sam program, trzy uruchomienia:

uruch. 1	-5	123	15	3	-10
uruch. 2	0	512	-1	8	0
uruch. 3	912	0	-77	2	19

(przykład — u Ciebie liczby będą inne)

04 Oba przypadki obok siebie

TAK JAK W NOTATKACH

```
int lista1[5];        // przypadek nr 1

int main(){
    int lista2[5];    // przypadek nr 2
}
```

Ta sama linijka — **int nazwa[5];** — a w środku coś zupełnie innego. Decyduje tylko **miejsce**, w którym ją napiszemy.

lista1 — globalna (poza main)

0	1	2	3	4
0	0	0	0	0

same zera

lista2 — lokalna (w main)

0	1	2	3	4
-5	123	15	3	-10

losowe śmieci

04 Co z tego wynika?

Zanim coś **odczytasz** z lokalnej tablicy bez listy, **najpierw coś do niej wpisz**.

`Chcę mieć same zera`

zrób tablicę **globalną** albo wyzeruj ją pętlą (rozdział 09).

`Zaraz i tak wczytam liczby`

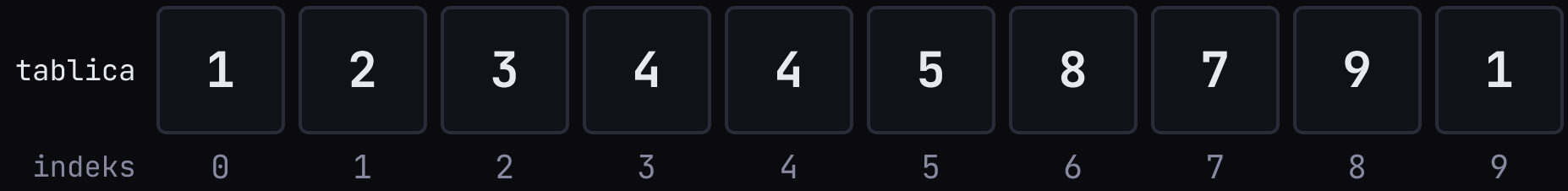
tablica **lokalna** jest OK — wczytane liczby nadpiszą śmieci (rozdział 10).

`Znam liczby od razu`

podaj **listę inicjalizacyjną** — w środku będzie dokładnie to, co wpiszesz.

05 Każda szufladka ma numer — indeks

```
int tablica[10] = {1, 2, 3, 4, 4, 5, 8, 7, 9, 1};
```



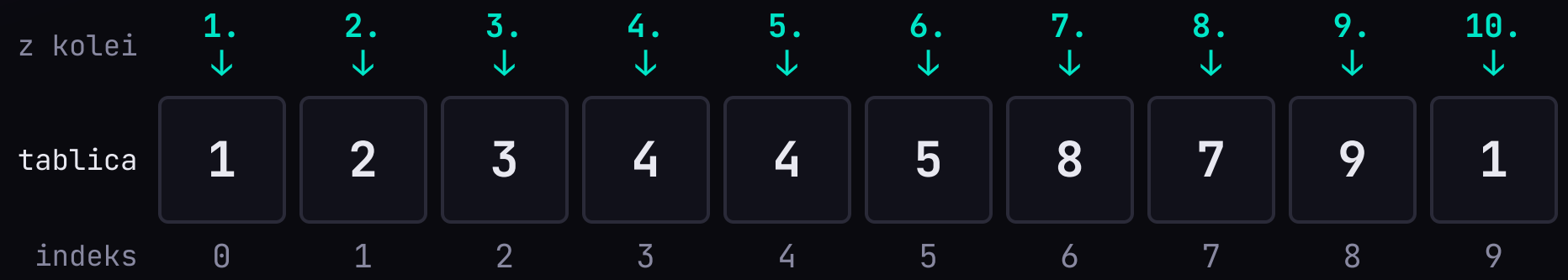
Numer szufladki nazywamy **indeksem**. Indeksy są napisane pod szufladkami: **0, 1, 2, ..., 9**.

Ważne: w C++ numerowanie zaczyna się od **0**, a nie od 1! Pierwsza szufladka ma indeks 0.

Intuicja: indeks mówi, **ile szufladek trzeba minąć** od początku. Do pierwszej nie mijamy żadnej → indeks 0.

05 Liczymy od zera — po kolei

Nad szufladkami: która to szufladka **z kolei**. Pod szufladkami: jej **indeks**.



Która z kolei	pierwsza	druga	trzecia	czwarta	…	dziesiąta
Indeks	**0**	**1**	**2**	**3**	**…**	**9**

Uwaga: indeks jest zawsze **o 1 mniejszy** niż numer „z kolei”. Dziesiąta (ostatnia) szufladka ma indeks **9**. Szufladki o indeksie 10 **nie ma**.

05 Zakres indeksów: od 0 do n-1

n → długość tablicy · zakres indeksów: 0 … n−1

Jeśli tablica ma **n** elementów, to pierwszy ma indeks **0**, a ostatni **n−1**. Indeksu **n** już **nie ma**.

Tablica	n (długość)	Indeksy	Ostatni indeks
`int tablica[10]`	10	0, 1, 2, …, 9	9
`int tablica[5]`	5	0, 1, 2, 3, 4	4
`int tablica[3]`	3	0, 1, 2	2
`int tablica[1]`	1	tylko 0	0

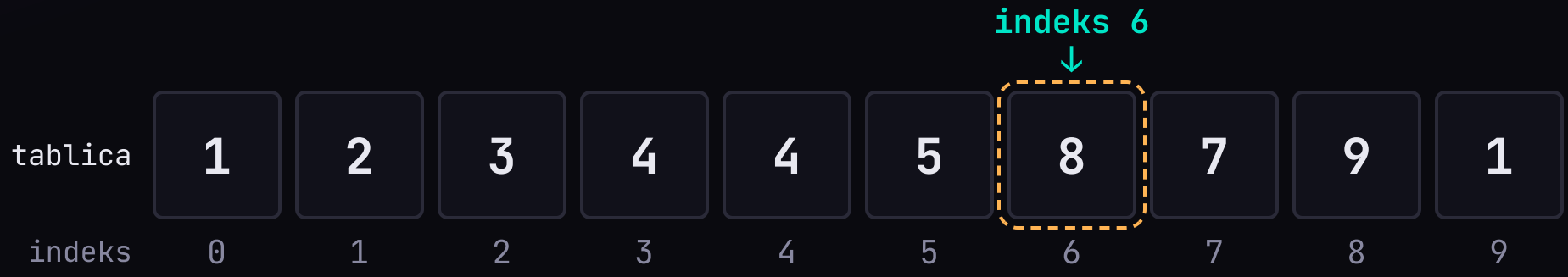
05 Dlaczego ostatni indeks to n-1?



Skojarzenie: jak piętra w bloku: **parter** to poziom 0, potem 1. piętro, 2. piętro… W bloku z 5 poziomami najwyższe jest **4. piętro**.

1. Wypisujemy indeksy tablicy o długości 5: **0, 1, 2, 3, 4**.
2. Liczymy, ile ich jest: **5** — tyle, ile szufladek. Zgadza się.
3. Ale zaczęliśmy od **0**, a nie od 1 — więc każdy numer jest „przesunięty o jeden w dół”.
4. Dlatego ostatni indeks to **5 − 1 = 4**. Ogólnie: **n − 1**.

05 Indeks to nie wartość!



W szufladce o **indeksie 6** leży **wartość 8**. Indeks mówi, **gdzie** szukać. Wartość mówi, **co** tam leży.

Co leży pod indeksem 3?

4

Co leży pod indeksem 4?

też 4 — wartości mogą się powtarzać

Pod jakim indeksem leży 9?

pod indeksem **8**

Pod jakim indeksem leży 1?

pod dwoma: **0** i **9**

06 Jak dostać się do jednej szufladki?

```
nazwa[indeks]
```

Piszemy nazwę tablicy, a zaraz za nią — w nawiasach kwadratowych — **indeks** szufladki. Na przykład `tablica[6]` czytamy: „tablica, szufladka o indeksie 6”.

TWORZENIE: W NAWIASIE ROZMIAR

```
int tablica[10] = {1, 2, 3, 4, 4, 5, 8, 7, 9, 1};
```

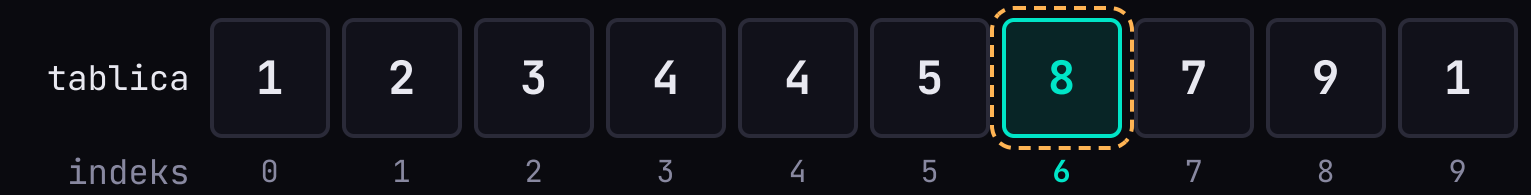
UŻYWANIE: W NAWIASIE INDEKS

```
cout << tablica[6];
```

Uwaga: nie myl dwóch nawiasów! Przy **tworzeniu** (`int tablica[10]`) w nawiasie stoi **rozmiar**. Przy **używaniu** (`tablica[6]`) w nawiasie stoi **indeks**.

06 Wypisujemy element o indeksie 6

```
int tablica[10] = {1, 2, 3, 4, 4, 5, 8, 7, 9, 1};
cout << tablica[6] << endl;
```



1. C++ bierze tablicę `tablica`.
2. Idzie do szufladki o indeksie **6** (mija 6 szufladek: 0, 1, 2, 3, 4, 5).
3. Wyjmuje to, co tam leży: **8**.
4. `cout` wypisuje 8, a `endl` przechodzi do nowej linii.

NA EKRANIE

```
8
```

06 Więcej przykładów odczytu

```cpp
int tablica[10] = {1, 2, 3, 4, 4, 5, 8, 7, 9, 1};

cout << tablica[0] << endl;   // pierwszy
cout << tablica[3] << endl;
cout << tablica[4] << endl;
cout << tablica[9] << endl;   // ostatni: n-1 = 9
```

NA EKRANIE

```
1
4
4
1
```



Każda linijka z cout wypisuje **jedną** szufladkę. Pod indeksem 3 i 4 leży ta sama wartość (4) — to nic dziwnego, to po prostu dwie różne szufladki z taką samą liczbą.

06 Cały program — odczyt jednego elementu

```cpp
#include <iostream>
using namespace std;

int main(){
    int tablica[10] = {1, 2, 3, 4, 4, 5, 8, 7, 9, 1};
    cout << tablica[6] << endl;
}
```

NA EKRANIE

```
8
```

Tablica stoi w środku main, więc jest **lokalna**. Ale ma **listę inicjalizacyjną** — dlatego nie ma w niej śmieci, tylko dokładnie nasze liczby.

06 Element tablicy działa jak zwykła zmienna

`tablica[2]` to po prostu jedna liczba `int`. Można jej użyć wszędzie tam, gdzie zwykłej zmiennej: przypisać, dodać, pomnożyć, wypisać.

```
int tablica[5] = {-2, 3, 5, 7, 9};

int x = tablica[2];                    // x = 5
cout << x << endl;
cout << tablica[0] + tablica[4] << endl;  // -2 + 9
cout << tablica[1] * 10 << endl;          // 3 * 10
```

NA EKRANIE

```
5
7
30
```



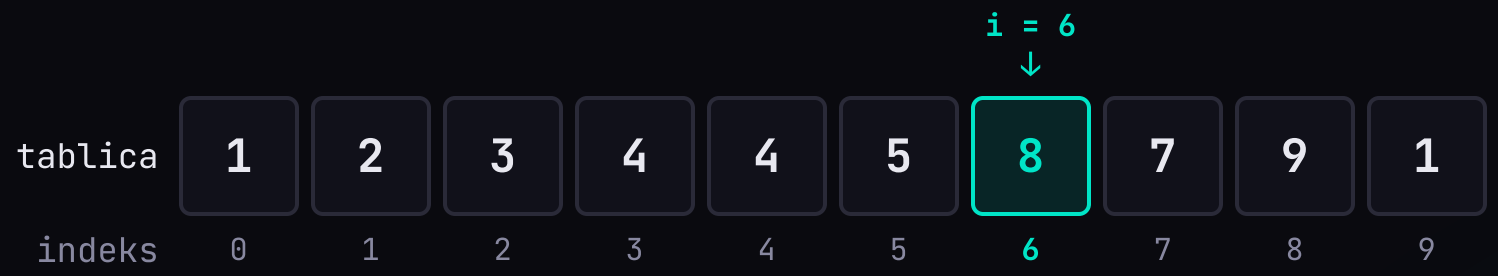
06 Indeks może być zmienną

W nawiasach kwadratowych nie musi stać liczba. Może tam stać **zmienna** — C++ weźmie jej **aktualną wartość**.

```cpp
int tablica[10] = {1, 2, 3, 4, 4, 5, 8, 7, 9, 1};
int i = 6;
cout << tablica[i] << endl;   // i = 6, więc tablica[6]
```

NA EKRANIE

```
8
```



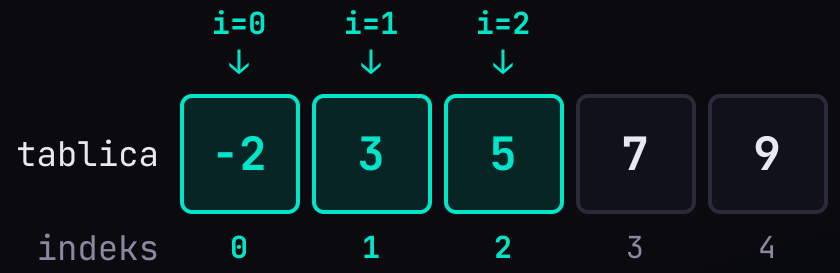
Ważne: to bardzo ważny krok! Dzięki niemu za chwilę **pętla for** przejdzie po całej tablicy — sama zmieniając `i`.

06 Zmieniamy i — zmienia się szufladka

```cpp
int tablica[5] = {-2, 3, 5, 7, 9};
int i = 0;
cout << tablica[i] << ' ';   // i = 0 → -2
i = 1;
cout << tablica[i] << ' ';   // i = 1 → 3
i = 2;
cout << tablica[i] << ' ';   // i = 2 → 5
```

NA EKRANIE

```
-2 3 5
```

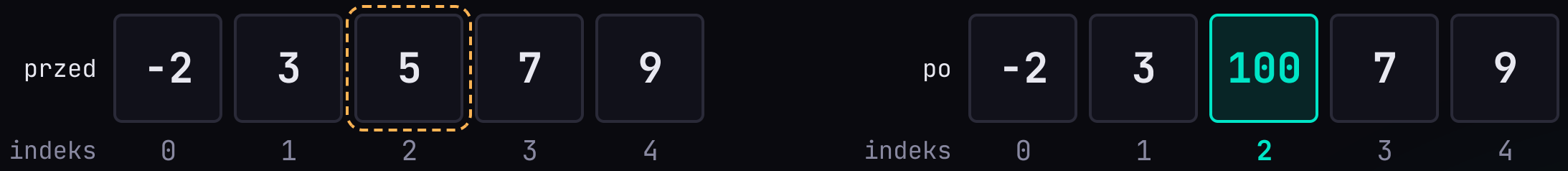


Linijka `cout << tablica[i] << ' ';` jest **za każdym razem taka sama** — a wypisuje coś innego, bo zmienia się `i`. Znak `' '` to spacja: oddziela liczby od siebie.

07 Wpisujemy nową wartość do szufladki

Do elementu tablicy wpisujemy wartość **tak samo jak do zmiennej** — znakiem `=`.

```
int tablica[5] = {-2, 3, 5, 7, 9};
tablica[2] = 100;
```

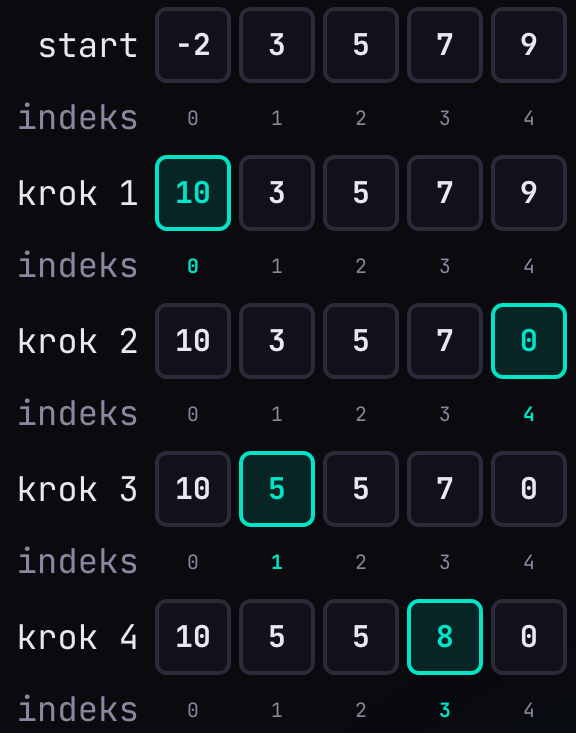


Stara wartość (**5**) znika — szufladka trzyma zawsze tylko jedną liczbę. Pozostałe szufladki się **nie zmieniają**.

07 Kilka zmian, jedna po drugiej

```
int tablica[5] = {-2, 3, 5, 7, 9};   // start

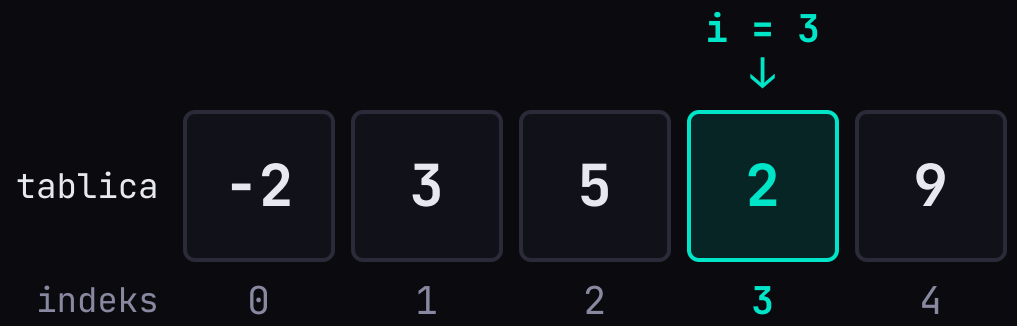
tablica[0] = 10;                     // krok 1
tablica[4] = 0;                      // krok 2
tablica[1] = tablica[2];             // krok 3
tablica[3] = tablica[3] + 1;         // krok 4
```



Krok 3: do szufladki 1 kopiujemy to, co leży w szufladce 2 (czyli 5). **Krok 4:** bierzemy starą wartość szufladki 3 (7), dodajemy 1 i wpisujemy z powrotem → 8.

07 Indeks w zmiennej — także przy wpisywaniu

```
int tablica[5] = {-2, 3, 5, 7, 9};
int i = 3;
tablica[i] = 2;    // i = 3, więc tablica[3] = 2
```



Tak samo jak przy odczycie — w nawiasie może stać zmienna. Ta linijka wpisuje 2 do szufladki, której indeks akurat siedzi w `i`.

Zapowiedź: za chwilę pętla for zrobi to za nas dla **każdego** i po kolei — i tak wypełni całą tablicę (rozdział 09).

08 Nie da się wyświetlić tablicy jednym `cout`!

TAK NIE ROBIMY

```
int tablica[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
cout << tablica << endl;   // to NIE wypisze liczb
```

NA EKRANIE — COŚ DZIWNEGO

```
0x7ffd5c3a1b20
```

Zamiast liczb pojawia się dziwny napis. To **adres w pamięci** — numer miejsca, w którym leży tablica. `cout` nie potrafi sam przejść po wszystkich szufladkach.

Nie da się wyświetlić tablicy jednym cout.

Zasada: każdy element tablicy trzeba wypisać **osobno**.

08 Sposób „na piechotę”: każdy element osobno

```
int tablica[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

cout << tablica[0] << ' ';
cout << tablica[1] << ' ';
cout << tablica[2] << ' ';
cout << tablica[3] << ' ';
cout << tablica[4] << ' ';
cout << tablica[5] << ' ';
cout << tablica[6] << ' ';
cout << tablica[7] << ' ';
cout << tablica[8] << ' ';
cout << tablica[9] << ' ';
cout << endl;
```

Działa! Każda linijka wypisuje jedną szufladkę i spację:

NA EKRANIE

```
1 2 3 4 5 6 7 8 9 10
```

Problem: przy 1000 elementów trzeba by napisać **1000 linijek**…

Zauważ: **wszystkie linijki są takie same**. Zmienia się tylko liczba w nawiasie: 0, 1, 2, …, 9 — po kolei, o jeden w górę.

08 Pętla for daje po kolei 0, 1, 2, …, 9

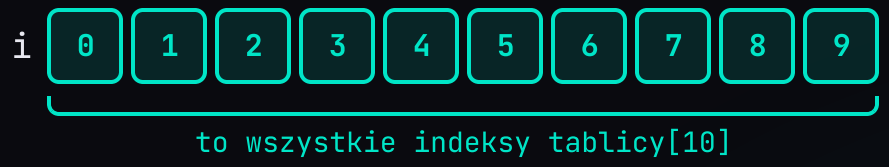
```
for(int i = 0; i < 10; i++){
    cout << i << ' ';
}
```

NA EKRANIE

```
0 1 2 3 4 5 6 7 8 9
```

Przypomnienie: ta pętla wypisuje kolejne wartości `i`. Spójrz — to dokładnie **wszystkie indeksy** tablicy o długości 10!

Fragment	Co znaczy
`int i = 0`	zaczynamy od 0 — to pierwszy indeks
`i < 10`	kręcimy, dopóki i < 10; ostatnie i to **9 = n−1**
`i++`	po każdym obrocie i rośnie o 1



08 Łączymy: `tablica[i]` w pętli

KOD Z NOTATEK

```
for(int i = 0; i < 10; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

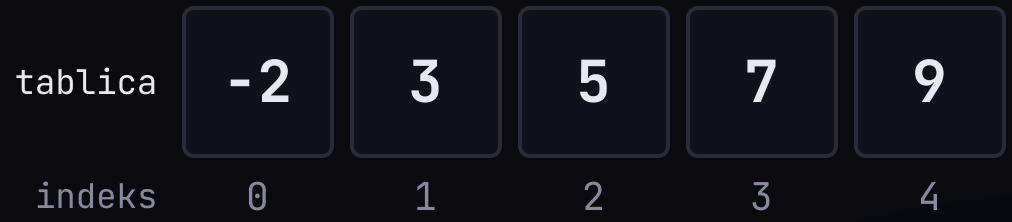
Zamiast samego `i` wypisujemy `tablica[i]` — element pod indeksem i. W każdym obrocie `i` jest inne, więc za każdym razem wypisujemy **inną szufladkę**.

Fragment	Co robi
`int i = 0`	start od indeksu 0 — pierwszy element
`i < 10`	koniec na i = 9 — ostatni element (n-1)
`i++`	przejście do następnej szufladki
`cout << tablica[i] << ' '`	wypisz element i spację
`cout << endl;`	po pętli, **raz**: nowa linia

08 Prześledźmy to powoli — na mniejszej tablicy

Żeby dokładnie zobaczyć, co robi pętla, weźmy tablicę z **5** elementami. Warunek to wtedy `i < 5`.

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```



Na kolejnych slajdach: **jeden slajd = jeden obrót pętli**. Patrz na zmienną `i`, podświetloną szufladkę i to, co pojawia się na ekranie.

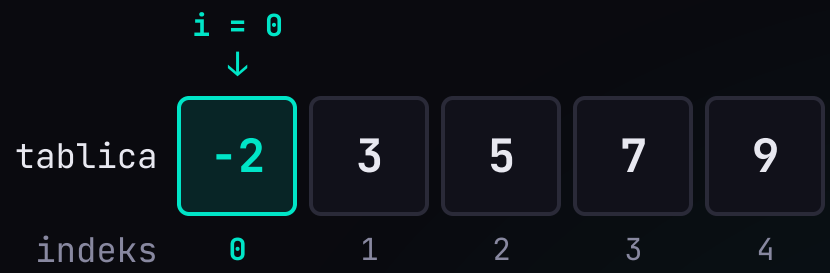
08 Obrót 1 z 5: i = 0

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

```
-2
```

i	0
warunek	`0 < 5` → **TAK** — wchodzimy do środka pętli
robimy	`cout << tablica[0] << ' '`
efekt	na ekran trafia **-2** i spacja
potem	`i++` → `i = 1`



08 Obrót 2 z 5: i = 1

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

```
-2 3
```

i	1
warunek	`1 < 5` → **TAK** — wchodzimy do środka pętli
robimy	`cout << tablica[1] << ' '`
efekt	na ekran trafia **3** i spacja
potem	`i++` → `i = 2`



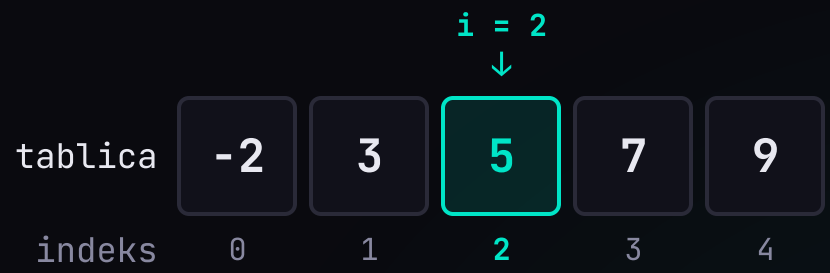
08 Obrót 3 z 5: i = 2

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

```
-2 3 5
```

i	2
warunek	`2 < 5` → **TAK** — wchodzimy do środka pętli
robimy	`cout << tablica[2] << ' '`
efekt	na ekran trafia **5** i spacja
potem	`i++` → `i = 3`



08 Obrót 4 z 5: i = 3

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

```
-2 3 5 7
```

i	3
warunek	`3 < 5` → **TAK** — wchodzimy do środka pętli
robimy	`cout << tablica[3] << ' '`
efekt	na ekran trafia **7** i spacja
potem	`i++` → `i = 4`



08 Obrót 5 z 5: i = 4

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

```
-2 3 5 7 9
```

i	4
warunek	`4 < 5` → **TAK** — wchodzimy do środka pętli
robimy	`cout << tablica[4] << ' '`
efekt	na ekran trafia **9** i spacja
potem	`i++` → `i = 5`



08 i = 5: koniec pętli

```
int tablica[5] = {-2, 3, 5, 7, 9};
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

```
-2 3 5 7 9
```

i	5
warunek	`5 < 5` → **NIE** — pętla się kończy
robimy	linijkę po pętli: `cout << endl;`
efekt	przejście do nowej linii



Szufladki o indeksie 5 **nie ma** — i dobrze, że pętla tu staje.

08 Cała pętla w jednej tabeli

i	i < 5 ?	Co się dzieje	Na ekranie po tym kroku
0	tak	wypisz `tablica[0]` → -2	`-2`
1	tak	wypisz `tablica[1]` → 3	`-2 3`
2	tak	wypisz `tablica[2]` → 5	`-2 3 5`
3	tak	wypisz `tablica[3]` → 7	`-2 3 5 7`
4	tak	wypisz `tablica[4]` → 9	`-2 3 5 7 9`
5	**nie**	koniec pętli → `cout << endl;`	`-2 3 5 7 9` + nowa linia

Pętla wykonała się **5 razy** — dla i = 0, 1, 2, 3, 4 — czyli dla **każdego indeksu** tablicy. Dla i = 5 warunek jest fałszywy i pętla się kończy.

08 Cały program z notatek

```cpp
#include <iostream>
using namespace std;

int main()
{
    int tablica[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    for(int i = 0; i < 10; i++){
        cout << tablica[i] << ' ';
    }
    cout << endl;
}
```

NA EKRANIE

```
1 2 3 4 5 6 7 8 9 10
```

Liczba **10** w warunku `i < 10` to długość tablicy. C++ policzył ją z listy: w klamrach jest 10 liczb.

Zapamiętaj: ten program to wzór, który będziesz pisać **bardzo często**.

08 Po co ta spacja?

BEZ SPACJI

```
for(int i = 0; i < 10; i++){
    cout << tablica[i];
}
```

NA EKRANIE

```
12345678910
```

ZE SPACJĄ

```
for(int i = 0; i < 10; i++){
    cout << tablica[i] << ' ';
}
```

NA EKRANIE

```
1 2 3 4 5 6 7 8 9 10
```

Bez spacji liczby się **sklejają** — nie wiadomo, czy to 1, 2, 3, … czy jedna wielka liczba 12345678910. Spacja oddziela kolejne elementy.

08 endl w środku pętli czy po pętli?

ENDL W ŚRODKU — PO KAŻDEJ LICZBIE

```
for(int i = 0; i < 5; i++){
    cout << tablica[i] << endl;
}
```

NA EKRANIE

```
-2
3
5
7
9
```

ENDL PO PĘTLI — RAZ, NA KOŃCU

```
for(int i = 0; i < 5; i++){
    cout << tablica[i] << ' ';
}
cout << endl;
```

NA EKRANIE

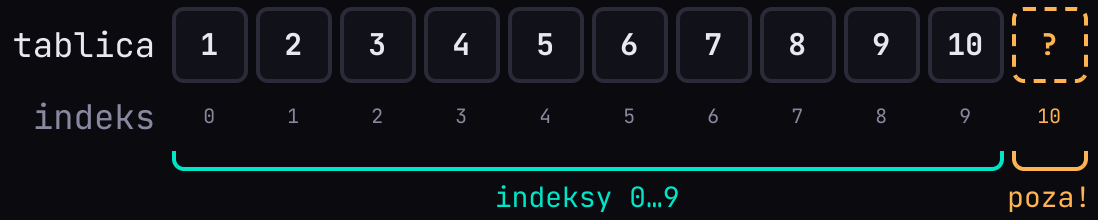
```
-2 3 5 7 9
```

Oba programy są poprawne — robią co innego. Najczęściej wypisujemy elementy **w jednej linii**, oddzielone spacjami, a `endl` dajemy **raz, po pętli**.

08 Uwaga na warunek: `i < 10`, a nie `i <= 10`

ŹLE

```
int tablica[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for(int i = 0; i <= 10; i++){   // i dojdzie do 10!
    cout << tablica[i] << ' ';
}
```



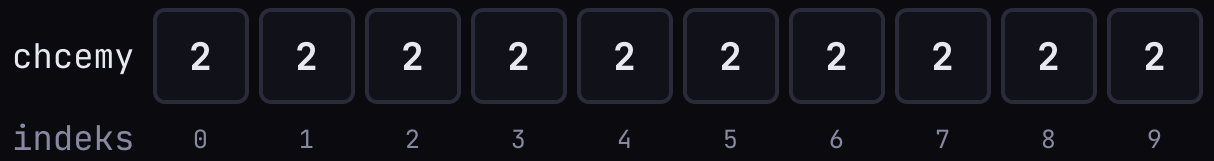
Uwaga: indeks **10 nie istnieje**. Program zagląda do pamięci **za tablicą** — wypisze śmieci albo się wysypie. W pętli po tablicy zawsze `i < n`.

NA EKRANIE (PRZYKŁAD)

```
1 2 3 4 5 6 7 8 9 10 32764
```

09 Zadanie: wpisz 2 do każdej szufladki

Mamy tablicę na 10 liczb i chcemy, żeby **w każdej szufladce była liczba 2**.



Można na piechotę — jak obok. Ale znowu: linijki są identyczne, zmienia się tylko indeks 0, 1, 2, …, 9.

> **Więc:** używamy pętli for — dokładnie jak przy wypisywaniu.

NA PIECHOTĘ

```
int tablica[10];

tablica[0] = 2;
tablica[1] = 2;
tablica[2] = 2;
// ... i tak dalej ...
tablica[9] = 2;
```

09 Pętla, która ustawia wartości

KOD Z NOTATEK

```
int tablica[10];
for(int i = 0; i < 10; i++){
    tablica[i] = 2;
}
```



W każdym obrocie pętli do szufladki o indeksie `i` wpisujemy 2. Pętla odwiedza **wszystkie** indeksy 0…9, więc **każda** szufladka dostaje 2 — stare śmieci znikają.

> Porównaj z wypisywaniem: pętla ta sama, zmienia się tylko środek — zamiast `cout << tablica[i]` jest `tablica[i] = 2`.

09 Krok po kroku: co się dzieje w tablicy

```
int tablica[5];
for(int i = 0; i < 5; i++){
    tablica[i] = 2;
}
```

> Pętla „przesuwa się” po tablicy od lewej do prawej. Po każdym obrocie **jedna szufladka więcej** ma wartość 2.

Gdy `i` dojdzie do 5, warunek `5 < 5` jest fałszywy — koniec.



09 Cały program: ustaw i wypisz

```cpp
#include <iostream>
using namespace std;

int main(){
    int tablica[10];
    for(int i = 0; i < 10; i++){   // pętla 1: ustaw
        tablica[i] = 2;
    }
    for(int i = 0; i < 10; i++){   // pętla 2: wypisz
        cout << tablica[i] << ' ';
    }
    cout << endl;
}
```

NA EKRANIE

```
2 2 2 2 2 2 2 2 2 2
```

Dwie **osobne** pętle, jedna po drugiej: najpierw cała tablica dostaje wartości, potem całą wypisujemy.

W obu pętlach możemy użyć nazwy `i` — każda pętla ma swoje własne `i`.

09 Zerowanie — sposób na śmieci

```
int main(){
    int lista2[5];                  // lokalna → śmieci
    for(int i = 0; i < 5; i++){
        lista2[i] = 0;              // każda = 0
    }
    // teraz lista2 = {0, 0, 0, 0, 0}
}
```



Połączenie: pamiętasz przypadek nr 2 z rozdziału 04? Lokalna tablica bez listy ma śmieci. Ta pętla ją **czyści** — od teraz są w niej same zera, tak jak w tablicy globalnej.

09 Wartość może zależeć od `i`

Po prawej stronie znaku `=` może stać nie tylko stała liczba, ale też coś z `i`. Wtedy każda szufladka dostaje **inną** wartość.

```
int tablica[5];
for(int i = 0; i < 5; i++){
    tablica[i] = i;
}
```

i	robimy	tablica po kroku
0	`tablica[0] = 0`	**0** ? ? ? ?
1	`tablica[1] = 1`	0 **1** ? ? ?
2	`tablica[2] = 2`	0 1 **2** ? ?
3	`tablica[3] = 3`	0 1 2 **3** ?
4	`tablica[4] = 4`	0 1 2 3 **4**

Wynik: **{0, 1, 2, 3, 4}** — każda szufladka trzyma swój własny indeks.

09 Więcej wzorów z i

```
int tablica[10];
for(int i = 0; i < 10; i++){
    tablica[i] = ...;   // co wpisać?
}
```

Ta sama pętla — zmieniamy tylko to, co stoi po znaku **=**. Pomarańczowa ramka pokazuje szufladkę **i = 3** — sprawdź ją w głowie!

`tablica[i] = i + 1;`

dla i = 3: 3 + 1 = 4

1	2	3	**4**	5	6	7	8	9	10

`tablica[i] = i * i;`

dla i = 3: 3 * 3 = 9

0	1	4	**9**	16	25	36	49	64	81

`tablica[i] = 10 - i;`

dla i = 3: 10 - 3 = 7

10	9	8	**7**	6	5	4	3	2	1

10 Jak wygląda wejście?

FORMAT WEJŚCIA

n
a_0 a_1 a_2 a_3 … a_{n-1}

W pierwszej linii jest **n** — ile liczb będzie dalej. W drugiej linii jest **n liczb** oddzielonych spacjami.

PRZYKŁAD

```
7
1 2 3 -5 8 -1 2
```

Tutaj n = 7, więc w drugiej linii jest 7 liczb.



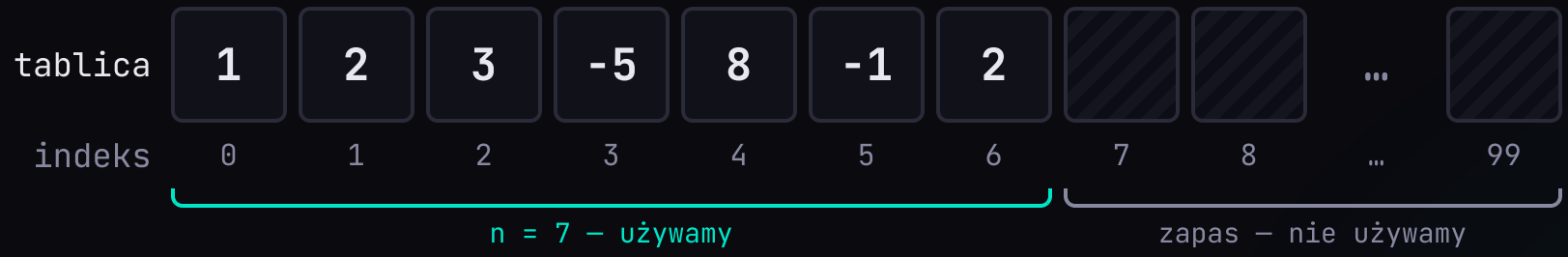
Liczby nazywamy a_0, a_1, …, a_{n-1} — mają numery **od 0 do n−1**, dokładnie jak indeksy tablicy! Liczba a_3 trafi do `tablica[3]`.

10 Problem: ile miejsca zrobić?

Program piszemy **zanim** zobaczymy wejście. Raz n = 7, innym razem n = 50. A rozmiar tablicy musimy ustalić **na początku** i nie można go zmienić (rozdział 02).

Rozwiązanie: bierzemy tablicę **z zapasem**. W treści zadania zawsze jest napisane, jakie może być **największe n**, np. „n ≤ 100”. Robimy tablicę co najmniej tak dużą:

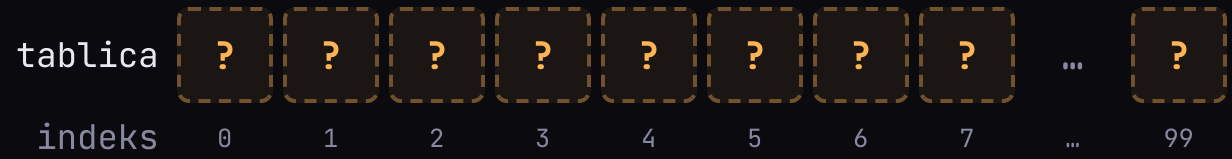
```
int tablica[100]; // 100 >= n
```



10 Budujemy program — krok 1: tablica z zapasem

```
int main()
{
    int tablica[100]; // 100 >= n
}
```

Tworzymy tablicę na **100** liczb.



Tablica jest **lokalna** (w main), więc w środku są śmieci. Ale to nic — zaraz nadpiszemy je wczytanymi liczbami.

10 Krok 2: zmienna n

```
int main()
{
    int tablica[100]; // 100 >= n
    int n;
}
```

Tworzymy zmienną **n**. Będzie pamiętać, **ile liczb** jest na wejściu.

n	? — na razie nic tam nie wpisaliśmy

n to **zwykła zmienna**, nie tablica — trzyma jedną liczbę.

10 Krok 3: wczytujemy n

```
int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

`cin >> n;` czyta **pierwszą** liczbę z wejścia — **7** — i wpisuje ją do n.

n	7

Efekt: teraz program wie, że za chwilę przyjdzie **7 liczb**.

10 Krok 4: pętla wczytująca

```
int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> tablica[i];
    }
}
```

Pętla przechodzi po indeksach **0, 1, …, n−1**. W każdym obrocie `cin >> tablica[i];` czyta **kolejną** liczbę z wejścia i wkłada ją do szufladki o indeksie i.

> **Uwaga:** w warunku jest `i < n`, a **nie** `i < 100`. Wczytujemy tyle liczb, ile jest naprawdę (n), a nie tyle, ile mamy miejsca.

10 Gotowy kod wczytywania — z notatek

```
int main()
{
    int tablica[100]; // 1. zapas: 100 >= n
    int n;            // 2. ile liczb
    cin >> n;         // 3. wczytaj n
    for(int i = 0; i < n; i++){
        cin >> tablica[i];   // 4. wczytaj n liczb
    }
}
```

1. Tablica **z zapasem** — większa niż największe n.
2. Zmienna `n` na liczbę elementów.
3. `cin >> n;` — najpierw wczytujemy, ile ich będzie.
4. Pętla `i < n` — wczytujemy n liczb, każdą do następnej szufladki.

10 Wczytujemy przykład — start: n = 7

```cpp
int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> tablica[i];
    }
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

robimy	`cin >> n;`
wczytane	**7**
n	**7**



Tablica nadal ma w środku śmieci (?). Za chwilę zacznie się pętla.

10 Obrót 1: i = 0

```cpp
int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> tablica[i];
    }
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

i	0
warunek	`0 < 7` → **TAK**
robimy	`cin >> tablica[0];`
wczytane	1 → trafia do szufladki 0
potem	`i++` → `i = 1`



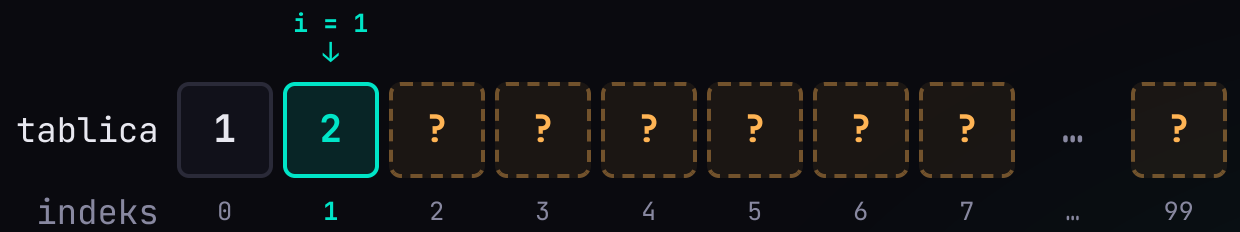
10 Obrót 2: i = 1

```
int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> tablica[i];
    }
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

i	1
warunek	`1 < 7` → **TAK**
robimy	`cin >> tablica[1];`
wczytane	2 → trafia do szufladki 1
potem	`i++` → `i = 2`



10 Kolejne obroty — w tabeli

i	i < 7 ?	wczytujemy	tablica po kroku (pierwsze 8 szufladek)
2	tak	`tablica[2] = 3`	1 2 **3** ? ? ? ? ?
3	tak	`tablica[3] = -5`	1 2 3 **-5** ? ? ? ?
4	tak	`tablica[4] = 8`	1 2 3 -5 **8** ? ? ?
5	tak	`tablica[5] = -1`	1 2 3 -5 8 **-1** ? ?
6	tak	`tablica[6] = 2`	1 2 3 -5 8 -1 **2** ?
7	**nie**	koniec pętli	1 2 3 -5 8 -1 2 ?

Szare **?** to śmieci — szufladki, do których jeszcze nic nie wczytaliśmy.

10 Koniec wczytywania: i = 7

```
int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> tablica[i];
    }
}
```

i	7
warunek	7 < 7 → NIE — koniec pętli
n	7
wczytano	wszystkie 7 liczb



WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

Gotowe: Liczby z wejścia są w szufladkach **0…6**, czyli **0…n−1**. Reszta tablicy to nieużywany zapas.

10 Dlaczego `i < n`, a nie `i < 100`?

ŹLE

```
for(int i = 0; i < 100; i++){   // chce 100 liczb!
    cin >> tablica[i];
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

Na wejściu jest tylko **7** liczb. Po ich wczytaniu pętla chce ósmą, dziewiątą… aż do setnej — których **nie ma**.

Skutek: program będzie **czekał** na liczby, które nigdy nie przyjdą (albo wczyta bzdury).

Dobrze: wczytujemy dokładnie tyle liczb, ile podano: `i < n`. Rozmiar tablicy (100) to tylko **miejsce**, a n to **ile naprawdę** używamy.

10 Wczytaj i wypisz — cały program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int tablica[100]; // 100 >= n
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> tablica[i];
    }
    for(int i = 0; i < n; i++){
        cout << tablica[i] << ' ';
    }
    cout << endl;
}
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

```
7
1 2 3 -5 8 -1 2
```

NA EKRANIE

```
1 2 3 -5 8 -1 2
```

W pętli wypisującej też jest `i < n` — wypisujemy tylko te n liczb, które wczytaliśmy, a nie śmieci z zapasu.

10 Ten sam program — różne wejścia

Nie zmieniamy **ani jednej linijki** kodu z poprzedniego slajdu — program sam dopasowuje się do n:

WEJŚCIE

```
3
10 -4 6
```

NA EKRANIE

```
10 -4 6
```

WEJŚCIE

```
1
-13
```

NA EKRANIE

```
-13
```

WEJŚCIE

```
10
1 2 3 4 4 5 8 7 9 1
```

NA EKRANIE

```
1 2 3 4 4 5 8 7 9 1
```

Działa dla każdego n od 1 do 100 — bo tyle mamy miejsca w tablicy.

10 Wariant z notatek: najpierw n, potem tablica

```
int n;
cin >> n;
int tablica[1000];
for(int i = 0; i < n; i++){
    cin >> tablica[i];
}
```

Kolejność nie ma znaczenia — można najpierw wczytać n, a dopiero potem stworzyć tablicę. Ważne, żeby tablica była **co najmniej tak duża jak największe możliwe n**.

Tu zapas to **1000** — wystarczy, gdy w zadaniu n ≤ 1000.

> **Pułapka:** kusi, żeby napisać `int tablica[n];`. Nie rób tak! W standardowym C++ rozmiar tablicy to **stała liczba** (np. 100, 1000) ustalona z góry — jak w najważniejszej zasadzie z rozdziału 02.

11 Pytanie 1: co wypisze program?

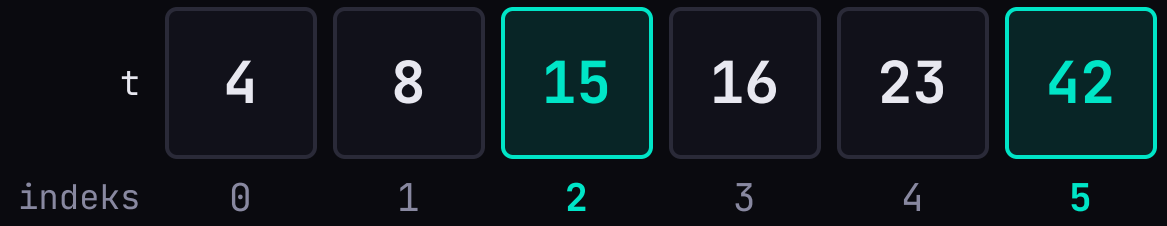
```
int t[] = {4, 8, 15, 16, 23, 42};
cout << t[2] << ' ' << t[5] << endl;
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 1: 15 42



1. Lista ma 6 liczb → rozmiar 6, indeksy **0…5**.
2. `t[2]` — trzecia szufladka → **15**.
3. `t[5]` — szósta, ostatnia (n−1 = 5) → **42**.

NA EKRANIE

```
15 42
```

11 Pytanie 2: globalna czy lokalna?

```
int a[3];

int main(){
    int b[3];
    cout << a[1] << endl;
    cout << b[1] << endl;
}
```

Co wypisze pierwsza linijka z cout? A co druga?

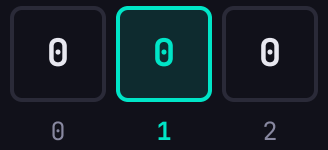
NA EKRANIE

```
?
?
```

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 2: zero i śmieć

a — globalna (poza main)



`a[1] = 0 — zawsze`

b — lokalna (w main)



`b[1] = śmieć — nie wiadomo, ile`

NA EKRANIE (DRUGA LICZBA — PRZYPADKOWA)

```
0
251
```

Uwaga: druga liczba może być **dowolna** i zmieniać się przy każdym uruchomieniu.

11 Pytanie 3: co wypisze program?

```
int t[5];
for(int i = 0; i < 5; i++){
    t[i] = i * 2;
}
for(int i = 0; i < 5; i++){
    cout << t[i] << ' ';
}
```

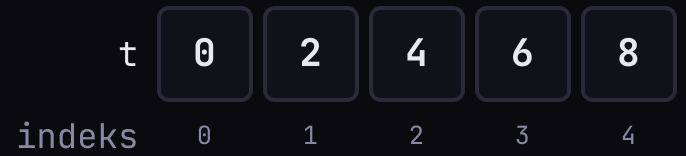
NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 3: 0 2 4 6 8

i	i * 2	wpisujemy
0	0 * 2 = 0	`t[0] = 0`
1	1 * 2 = 2	`t[1] = 2`
2	2 * 2 = 4	`t[2] = 4`
3	3 * 2 = 6	`t[3] = 6`
4	4 * 2 = 8	`t[4] = 8`



NA EKRANIE

```
0 2 4 6 8
```

Pierwsza pętla **ustawia**, druga **wypisuje**.

11 Pytanie 4: wczytywanie i indeks n-1

```cpp
int tablica[100];
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> tablica[i];
}
cout << tablica[0] + tablica[n-1] << endl;
```

WEJŚCIE (WPISUJEMY Z KLAWIATURY)

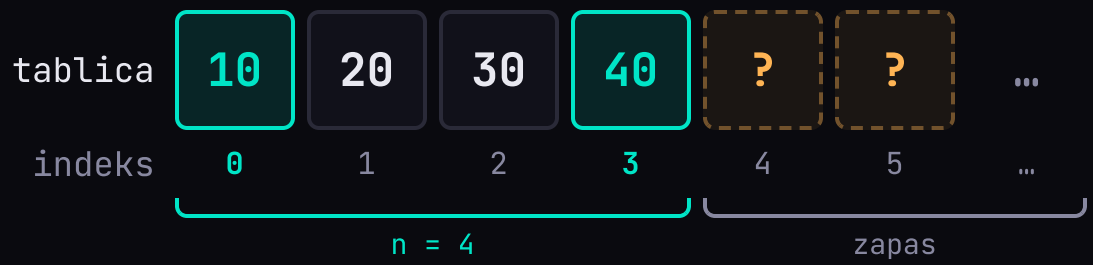
```
4
10 20 30 40
```

NA EKRANIE

```
?
```

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 4: 50



1. n = 4, wczytane liczby leżą pod indeksami **0…3**.
2. `tablica[0]` = **10** — pierwsza liczba.
3. n−1 = 4−1 = 3, więc `tablica[n-1]` = `tablica[3]` = **40** — ostatnia.
4. 10 + 40 = **50**.

NA EKRANIE

```
50
```

11 Pytanie 5: znajdź dwa błędy

```
int t[5] = {1, 2, 3, 4, 5};
for(int i = 1; i <= 5; i++){
    cout << t[i] << ' ';
}
```

Program miał wypisać:

OCZEKIWANY WYNIK

```
1 2 3 4 5
```

Co jest nie tak?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 5: start od 0 i warunek <

ŹLE

```
for(int i = 1; i <= 5; i++){
    cout << t[i] << ' ';
}
```

NA EKRANIE

```
2 3 4 5 32764
```

DOBRZE

```
for(int i = 0; i < 5; i++){
    cout << t[i] << ' ';
}
```

NA EKRANIE

```
1 2 3 4 5
```

01 **Start od i = 1** — pomija `t[0]`, czyli pierwszą liczbę (1).

02 **Warunek i <= 5** — czyta `t[5]`, którego nie ma (zakres to 0…4) → śmieć.

12 Najczęstsze błędy (1/2)

01 **Liczenie od 1.** `tablica[1]` to **drugi** element. Pierwszy to `tablica[0]`.

02 **Indeks n.** W tablicy `int t[10]` nie ma `t[10]` — ostatni to `t[9]`. W pętli piszemy `i < n`, a nie `i <= n`.

03 `cout << tablica;` wypisuje adres w pamięci, a nie liczby. Wypisuj pętlą — element po elemencie.

04 **Czytanie śmieci.** Lokalna tablica bez listy ma losowe wartości. Najpierw ją ustaw (pętla) albo wczytaj, dopiero potem z niej czytaj.

12 Najczęstsze błędy (2/2)

01 `int tablica[];` **bez listy.** Puste nawiasy działają tylko z listą inicjalizacyjną — inaczej C++ nie zna rozmiaru.

02 **Za mała tablica.** Jeśli n może być 1000, a tablica ma 100 miejsc — program pisze poza tablicą. Zapas: rozmiar ≥ największe n.

03 **Pętla do rozmiaru zamiast do n.** Przy wczytywaniu `i < n`, a nie `i < 100` — inaczej program czeka na liczby, których nie ma.

04 **Za dużo liczb w klamrach.** `int t[3] = {1, 2, 3, 4};` — 4 liczby nie zmieszczą się w 3 szufladkach → błąd kompilacji.

13 Ściąga: tworzenie i indeksy

Tworzenie z rozmiarem

`int t[5] = {-2, 3, 5, 7, 9};`

Rozmiar policzy C++

`int t[] = {2, 5, 7};` → 3 szufladki

Globalna bez listy (poza main)

`int t[5];` → same zera

Lokalna bez listy (w main)

`int t[5];` → losowe śmieci

Rozmiar

ustalamy na początku — **nie da się go zmienić**

Indeksy

od **0** do **n-1**

Odczyt jednego elementu

`cout << t[2] << endl;`

Zmiana jednego elementu

`t[2] = 100;`

Indeks w zmiennej

`t[i]` — bierze aktualne i

Cała tablica jednym cout?

NIE — tylko pętlą

13 Ściąga: trzy pętle po tablicy

WYPISZ CAŁĄ TABLICĘ

```
for(int i = 0; i < n; i++){
    cout << t[i] << ' ';
}
cout << endl;
```

USTAW WARTOŚĆ W KAŻDEJ SZUFLADCE

```
for(int i = 0; i < n; i++){
    t[i] = 2;
}
```

WCZYTAJ N LICZB

```
int t[100]; // 100 >= n
int n;
cin >> n;
for(int i = 0; i < n; i++){
    cin >> t[i];
}
```

Wspólny wzór: Wszystkie trzy wyglądają tak samo: `for(int i = 0; i < n; i++)`. Zmienia się tylko to, co robimy w środku z **t[i]**.

13 Zadania na koniec — Twoja kolej

1. Stwórz tablicę `{5, 10, 15, 20}` i wypisz jej trzeci element. (Uwaga na indeks!)
2. Stwórz lokalną tablicę na 8 liczb, wpisz pętlą do każdej szufladki 7 i wypisz całą tablicę w jednej linii.
3. Wczytaj n (n ≤ 100), a potem n liczb. Wypisz je w jednej linii, oddzielone spacjami.
4. Wczytaj n i n liczb. Wypisz pierwszą i ostatnią z nich. Podpowiedź: indeksy 0 i n−1.
5. Wypełnij tablicę 10-elementową tak, żeby było w niej `{0, 3, 6, 9, …, 27}`, i ją wypisz. Podpowiedź: `tablica[i] = i * …`

// KONIEC LEKCJI 7

Umiesz już korzystać z tablic!

Tworzenie, lista inicjalizacyjna, tablice globalne i lokalne, indeksy od 0 do n-1, wypisywanie, ustawianie i wczytywanie — wszystko krok po kroku. Teraz zrób zadania z poprzedniego slajdu.

